



RAPPORT

Consolidation programmation python

Réalisé Par :

- Mouhamadou Falilou Sarr

Sous la direction :

- PR Samuel Ouya

PARTIE I: Création de la base de données

- D'abord on se connecte au serveur mysql :

```
root@ubuntu:/home/falilou# mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 16
Server version: 8.0.33-0ubuntu0.20.04.4 (Ubuntu)

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> █
```

- Créons et utilisons la base de données python_api :

```
mysql> create database python_api;
Query OK, 1 row affected (0,15 sec)

mysql> use python_api
Database changed
mysql> █
```

- Créons la table banque :

```
mysql> create table banque(id int primary key auto_increment, nom varchar(30), prenom varchar(30), code varchar(30), numcompte varchar(30), solde varchar(30));
Query OK, 0 rows affected (0,06 sec)

mysql> █
```

- Créons l'utilisateur ghost :

```
mysql> create user ghost identified by 'passer';
Query OK, 0 rows affected (0,05 sec)

mysql> █
```

- Donnons tous les droits à ghost dans la base de données python_api :

```
mysql> grant all privileges on python_api.* to ghost;
Query OK, 0 rows affected (0,01 sec)

mysql> █
```

- Appliquons les privilèges :

```
mysql> flush privileges;
Query OK, 0 rows affected (0,01 sec)

mysql> █
```

PARTIE II : Utilisation de PyMySQL

- Créons un environnement virtuel qu'on appellera **my_venv** :

```
python3 -m venv my_venv : commande exécutable
root@ubuntu-fs:/usr/lib/cgi-bin# python3 -m venv my_venv
root@ubuntu-fs:/usr/lib/cgi-bin# █
```

- Entrons dans notre environnement virtuel :

```
root@ubuntu-fs:/usr/lib/cgi-bin# source my_venv/bin/activate
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# █
```

- Installons le module **pymysql** dans notre environnement virtuel :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# pip install pymysql
Collecting pymysql
  Using cached PyMySQL-1.1.0-py3-none-any.whl (44 kB)
Installing collected packages: pymysql
Successfully installed pymysql-1.1.0
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# █
```

- Installons le module **cryptography** dans notre environnement virtuel :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# pip install cryptography
Collecting cryptography
  Downloading cryptography-41.0.7-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (4.4 MB)
    |████████████████████| 4.4 MB 11.3 MB/s
Collecting cffi>=1.12
  Downloading cffi-1.16.0-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (444 kB)
    |████████████████████| 444 kB 10.9 MB/s
Collecting pycparser
  Downloading pycparser-2.21-py2.py3-none-any.whl (118 kB)
    |████████████████████| 118 kB 18.4 MB/s
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-1.16.0 cryptography-41.0.7 pycparser-2.21
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# █
```

- Ecrivons un script python avec le module pymysql **apicrud.py** qui nous permettra de faire des insertions, de faire des versements, de faire des retraits, de supprimer et de lister les enregistrements de ma base de données :

```

GNU nano 4.8                                                                    apicrud.py
!my_venv/bin/python3
import pymysql

def database():
    global conn, cursor
    conn = pymysql.connect(host="localhost", user='ghost', passwd='passer', database='python_api')
    cursor = conn.cursor()

def insertion(prenom, nom, code, numcompte, solde):
    database()
    sql = "insert into banque(prenom, nom, code, numcompte, solde) values (%s, %s, %s, %s, %s)"
    val = (prenom, nom, code, numcompte, solde)
    cursor.execute(sql, val)
    conn.commit()
    conn.close()
    print ("Insertion reussie")

def lecture():
    database()
    sql = "select * from banque"
    cursor.execute(sql)
    tab = cursor.fetchall()
    conn.close()
    print (tab)

def versement(numcompte, montant):
    database()
    sql1 = "select solde from banque where numcompte=%s"
    val1 = (numcompte,)
    cursor.execute(sql1, val1)
    tab1 = cursor.fetchall()
    if len(tab1) > 0:
        newsolde = float(tab1[0][0]) + float(montant)
        sql2 = "update banque set solde=%s where numcompte=%s"
        val2 = (newsolde, numcompte)
        cursor.execute(sql2, val2)
        conn.commit()
        message = (f"Versement de la somme de {montant} sur le compte {numcompte} effectue avec succes")
    else:
        message = ("compte inexistant")
    conn.close()
    print (message)

def retrait(numcompte, montant):
    database()
    sql1 = "select solde from banque where numcompte=%s"
    val1 = (numcompte,)
    cursor.execute(sql1, val1)
    tab2 = cursor.fetchall()
    if len(tab2) > 0:
        nouveausolde = float(tab2[0][0]) - float(montant)
        sql2 = "update banque set solde=%s where numcompte=%s"
        val2 = (nouveausolde, numcompte)
        cursor.execute(sql2, val2)
        conn.commit()
        message = (f"retrait de la somme de {montant} sur le compte {numcompte} effectue avec succes")
    else:
        message = ("Compte inexistant")
    conn.close()
    print (message)

def suppression(prenom, nom, code, numcompte):
    database()
    sql = "delete from banque where prenom=%s and nom=%s and code=%s and numcompte=%s"
    val = (prenom, nom, code, numcompte)
    cursor.execute(sql, val)
    conn.commit()
    conn.close()
    print (f"Le compte de {prenom} {nom} dont le numero est {numcompte} est supprime avec succes")

```

- Rendons notre script exécutable :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# chmod 755 apicrud.py
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin#
```

- Testons notre script en insérant Falilou en ajoutant la ligne ci dessous a notre script :

```
insertion("Falilou","Sarr","2407","1000","5000000")
```

- Lançons le script :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# python apicrud.py
Insertion reussie
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin#
```

- Vérifions dans la base de données si Falilou est bien inséré :

```
mysql> select * from banque;
+-----+-----+-----+-----+-----+-----+
| id | nom | prenom | code | numcompte | solde |
+-----+-----+-----+-----+-----+-----+
| 2 | Sarr | Falilou | 2407 | 1000 | 5000000 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0,00 sec)

mysql>
```

- Faisons un versement à Falilou en ajoutant la ligne ci dessous a notre script :

```
#insertion("Falilou","Sarr","2407","1000","5000000")
versement("1000","300000")
```

- Lançons le script :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# python apicrud.py
Versement de la somme de 300000 sur le compte 1000 effectue avec succes
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin#
```

- Vérifions dans la base de données si le versement a été effectué :

```
mysql> select * from banque;
+-----+-----+-----+-----+-----+-----+
| id | nom | prenom | code | numcompte | solde |
+-----+-----+-----+-----+-----+-----+
| 2 | Sarr | Falilou | 2407 | 1000 | 5000000 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0,00 sec)

mysql> select * from banque;
+-----+-----+-----+-----+-----+-----+
| id | nom | prenom | code | numcompte | solde |
+-----+-----+-----+-----+-----+-----+
| 2 | Sarr | Falilou | 2407 | 1000 | 5300000 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0,00 sec)

mysql> █
```

- Effectuons un retrait en ajoutant la ligne ci dessous a notre script :

```
#insertion("Falilou","Sarr","2407","1000","5000000")
#versement("1000","300000")
retrait("1000","705000")
```

- Lançons le script :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# python apicrud.py
retrait de la somme de 705000 sur le compte 1000 effectue avec succes
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# █
```

- Vérifions dans la base de données si le retrait a été effectué :

```
mysql> select * from banque;
+-----+-----+-----+-----+-----+-----+
| id | nom | prenom | code | numcompte | solde |
+-----+-----+-----+-----+-----+-----+
| 2 | Sarr | Falilou | 2407 | 1000 | 5300000 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0,00 sec)

mysql> select * from banque;
+-----+-----+-----+-----+-----+-----+
| id | nom | prenom | code | numcompte | solde |
+-----+-----+-----+-----+-----+-----+
| 2 | Sarr | Falilou | 2407 | 1000 | 4595000 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0,00 sec)

mysql> █
```

- Supprimons Falilou en ajoutant la ligne ci dessous a notre script :

```
#insertion("Falilou","Sarr","2407","1000","5000000")
#versement("1000","300000")
#retrait("1000","705000")
suppression("Falilou","Sarr","2407","1000")
█
```

- Lançons le script :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# python apicrud.py
Le compte de Falilou Sarr dont le numero est 1000 est supprime avec succes
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# █
```

- Vérifions dans la base de données si Falilou est supprimé :

```
mysql> select * from banque;
Empty set (0,00 sec)

mysql> █
```

- Créons Falilou de nouveau et listons la base de données en ajoutant la ligne ci dessous a notre script:

```
#insertion("Falilou","Sarr","2407","1000","5000000")
#versement("1000","300000")
#retrait("1000","705000")
#suppression("Falilou","Sarr","2407","1000")
lecture()
█
```

- Lançons le script :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# python apicrud.py
((3, 'Sarr', 'Falilou', '2407', '1000', '5000000'),)
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# █
```


PARTIE III : Création de l'Api avec Flask

- Installons le paquet **flask** dans notre environnement virtuel :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# pip install flask
Collecting flask
  Using cached flask-3.0.0-py3-none-any.whl (99 kB)
Collecting blinker>=1.6.2
  Using cached blinker-1.7.0-py3-none-any.whl (13 kB)
Collecting itsdangerous>=2.1.2
  Using cached itsdangerous-2.1.2-py3-none-any.whl (15 kB)
Collecting importlib-metadata>=3.6.0; python_version < "3.10"
  Using cached importlib_metadata-7.0.0-py3-none-any.whl (23 kB)
Collecting click>=8.1.3
  Using cached click-8.1.7-py3-none-any.whl (97 kB)
Collecting Jinja2>=3.1.2
  Using cached Jinja2-3.1.2-py3-none-any.whl (133 kB)
Collecting Werkzeug>=3.0.0
  Using cached werkzeug-3.0.1-py3-none-any.whl (226 kB)
Collecting zipp>=0.5
  Using cached zipp-3.17.0-py3-none-any.whl (7.4 kB)
Collecting MarkupSafe>=2.0
  Using cached MarkupSafe-2.1.3-cp38-cp38-manylinux_2_17_x86_64.m
Installing collected packages: blinker, itsdangerous, zipp, importlib-metadata, click, Jinja2, Werkzeug, MarkupSafe, flask
Successfully installed Jinja2-3.1.2 MarkupSafe-2.1.3 Werkzeug-3.0.1 flask-3.0.0 itsdangerous-2.1.2 zipp-3.17.0
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin#
```

- Installons le paquet **flask-Restx** dans notre environnement virtuel :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# pip install flask-Restx
Collecting flask-Restx
  Using cached flask_restx-1.3.0-py2.py3-none-any.whl (2.8 MB)
Collecting jsonschema
  Using cached jsonschema-4.20.0-py3-none-any.whl (84 kB)
Collecting aniso8601>=0.82
  Using cached aniso8601-9.0.1-py2.py3-none-any.whl (52 kB)
Collecting pytz
  Using cached pytz-2023.3.post1-py2.py3-none-any.whl (502 kB)
Requirement already satisfied: werkzeug<2.0.0 in ./my_venv/lib/python3.8/site-packages (from flask-Restx)
Requirement already satisfied: Flask!=2.0.0,>=0.8 in ./my_venv/lib/python3.8/site-packages (from flask-Restx)
Collecting importlib-resources
  Using cached importlib_resources-6.1.1-py3-none-any.whl (33 kB)
Collecting attrs>=22.2.0
  Using cached attrs-23.1.0-py3-none-any.whl (61 kB)
Collecting rpds-py>=0.7.1
  Using cached rpds_py-0.13.2-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.2 MB)
Collecting jsonschema-specifications>=2023.03.6
  Using cached jsonschema_specifications-2023.11.2-py3-none-any.whl (17 kB)
Collecting referencing>=0.28.4
  Using cached referencing-0.32.0-py3-none-any.whl (26 kB)
Collecting pkgutil_resolve_name>=1.3.10; python_version < "3.9"
  Using cached pkgutil_resolve_name-1.3.10-py3-none-any.whl (4.7 kB)
Requirement already satisfied: MarkupSafe>=2.1.1 in ./my_venv/lib/python3.8/site-packages (from flask-Restx)
Requirement already satisfied: Jinja2>=3.1.2 in ./my_venv/lib/python3.8/site-packages (from flask-Restx)
Requirement already satisfied: blinker>=1.6.2 in ./my_venv/lib/python3.8/site-packages (from flask-Restx)
Requirement already satisfied: importlib-metadata>=3.6.0; python_version < "3.10" in ./my_venv/lib/python3.8/site-packages (from flask-Restx)
Requirement already satisfied: click>=8.1.3 in ./my_venv/lib/python3.8/site-packages (from flask-Restx)
Requirement already satisfied: itsdangerous>=2.1.2 in ./my_venv/lib/python3.8/site-packages (from flask-Restx)
Requirement already satisfied: zipp>=3.1.0; python_version < "3.10" in ./my_venv/lib/python3.8/site-packages (from flask-Restx)
Installing collected packages: importlib-resources, attrs, rpds-py, referencing, jsonschema-specifications, jsonschema, flask-Restx
Successfully installed aniso8601-9.0.1 attrs-23.1.0 flask-Restx-1.3.0 importlib-resources-6.1.1 jsonschema-4.20.0 jsonschema-specifications-2023.11.2 referencing-0.32.0 rpds-py-0.13.2
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin#
```

- Installons le paquet **jsonify** :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# pip install jsonify
Processing /root/.cache/pip/wheels/ee/c6/ef/643adfbfd90fefe0640cb6b
Installing collected packages: jsonify
Successfully installed jsonify-0.5
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin#
```


- Installons le paquet **crud** :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# pip install crud
Processing /root/.cache/pip/wheels/02/81/30/41442a5ca423b58af860
Installing collected packages: crud
Successfully installed crud-0.1
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin#
```

- Installons le paquet **requests** :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# pip install requests
Collecting requests
  Using cached requests-2.31.0-py3-none-any.whl (62 kB)
Collecting charset-normalizer<4,>=2
  Using cached charset-normalizer-3.3.2-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (141 kB)
Collecting urllib3<3,>=1.21.1
  Using cached urllib3-2.1.0-py3-none-any.whl (104 kB)
Collecting idna<4,>=2.5
  Using cached idna-3.6-py3-none-any.whl (61 kB)
Collecting certifi>=2017.4.17
  Using cached certifi-2023.11.17-py3-none-any.whl (162 kB)
Installing collected packages: charset-normalizer, urllib3, idna, certifi, requests
Successfully installed certifi-2023.11.17 charset-normalizer-3.3.2 idna-3.6 requests-2.31.0 urllib3-2.1.0
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin#
```

- Modifions notre script **apicrud.py** en utilisant jsonify pour retourner les messages et ajoutons les fonctions de mise à jour , de transfert et de consultation de solde :

```
GNU nano 4.8 apicrud.py
#!/my_venv/bin/python3
import pymysql
from flask import jsonify

def database():
    global conn, cursor
    conn = pymysql.connect(host="localhost", user='ghost', passwd='passer', database='python_api')
    cursor = conn.cursor()

def insertion(prenom, nom, code, numcompte, solde):
    database()
    sql = "insert into banque(prenom, nom, code, numcompte, solde) values (%s, %s, %s, %s, %s)"
    val = (prenom, nom, code, numcompte, solde)
    cursor.execute(sql, val)
    conn.commit()
    conn.close()
    return ("Insertion reussie")

def lecture():
    database()
    sql = "select * from banque"
    cursor.execute(sql)
    tab = cursor.fetchall()
    conn.close()
    return (tab)

def update(numcompte, solde):
    database()
    sql = "update banque set solde=%s where numcompte=%s"
    val = (solde, numcompte)
    cursor.execute(sql, val)
    conn.commit()
    return (f"Le solde du compte {numcompte} a ete mise a jour")
```

```

def versement(numcompte, montant):
    database()
    sql1 = "select solde from banque where numcompte=%s"
    val1 = (numcompte,)
    cursor.execute(sql1, val1)
    tab1 = cursor.fetchall()
    if len(tab1) > 0:
        newsolde = float(tab1[0][0]) + float(montant)
        sql2 = "update banque set solde=%s where numcompte=%s"
        val2 = (newsolde, numcompte)
        cursor.execute(sql2, val2)
        conn.commit()
        message = (f"Versement de la somme de {montant} sur le compte {numcompte} effectue avec succes")
    else:
        message = ("compte inexistant")
    conn.close()
    return (message)

def retrait(numcompte, montant):
    database()
    sql1 = "select solde from banque where numcompte=%s"
    val1 = (numcompte,)
    cursor.execute(sql1, val1)
    tab2 = cursor.fetchall()
    if len(tab2) > 0:
        nouveausolde = float(tab2[0][0]) - float(montant)
        sql2 = "update banque set solde=%s where numcompte=%s"
        val2 = (nouveausolde, numcompte)
        cursor.execute(sql2, val2)
        conn.commit()
        message = (f"retrait de la somme de {montant} sur le compte {numcompte} effectue avec succes")
    else:
        message = ("Compte inexistant")
    conn.close()
    return (message)

def suppression(prenom, nom, code, numcompte):
    database()
    sql = "delete from banque where prenom=%s and nom=%s and code=%s and numcompte=%s"
    val = (prenom, nom, code, numcompte)
    cursor.execute(sql, val)
    conn.commit()
    conn.close()
    return (f"Le compte de {prenom} {nom} dont le numero est {numcompte} est supprime avec succes")

def transfert(numcomptedonneur, numcomptereceveur, montant):
    database()
    sql_don = "update banque set solde = solde - %s where numcompte = %s"
    val_don = (float(montant), numcomptedonneur)
    cursor.execute(sql_don, val_don)
    sql_rec = "update banque set solde = solde + %s where numcompte = %s"
    val_rec = (float(montant), numcomptereceveur)
    cursor.execute(sql_rec, val_rec)
    conn.commit()
    conn.close()
    return (f"Transfert de {montant} effectué avec succès")

def consultation(numcompte, code):
    database()
    sql = "select solde from banque where numcompte=%s and code=%s "
    val = (numcompte, code)
    cursor.execute(sql, val)
    result = cursor.fetchone()
    if result is not None:
        solde = result[0]
        return (f"Le solde du compte {numcompte} est : {solde}")
    else:
        return (None)

```

- Effectuons un transfert du compte 1000 vers 1001 en ajoutant la ligne ci dessous a notre script :

```
#insertion("Falilou","Sarr","2407","1000","5000000")
#insertion("Mouhamed","Wade","1301","1001","3800000")
#versement("1000","3000000")
#retrait("1000","7050000")
#suppression("Falilou","Sarr","2407","1000")
#lecture()
transfert("1000","1001","200000")
```

- Lançons le script :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# python3 apicrud.py
Transfert de 200000 effectué avec succès
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin#
```

- Vérifions dans la base de données si le transfert est fait :

```
mysql> select * from banque;
+-----+-----+-----+-----+-----+-----+
| id | nom | prenom | code | numcompte | solde |
+-----+-----+-----+-----+-----+
| 1 | Sarr | Falilou | 2407 | 1000 | 5000000 |
| 2 | Wade | Mouhamed | 1301 | 1001 | 3800000 |
+-----+-----+-----+-----+-----+
2 rows in set (0,00 sec)

mysql> select * from banque;
+-----+-----+-----+-----+-----+-----+
| id | nom | prenom | code | numcompte | solde |
+-----+-----+-----+-----+-----+
| 1 | Sarr | Falilou | 2407 | 1000 | 4800000 |
| 2 | Wade | Mouhamed | 1301 | 1001 | 4000000 |
+-----+-----+-----+-----+-----+
2 rows in set (0,00 sec)

mysql>
```

- Consultons le solde de Falilou en ajoutant la ligne ci dessous a notre script :

```
#lecture()
#transfert("1000","1001","200000")
consultation("1000","2407")
```

- Lançons le script :

```
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin# python3 apicrud.py
Le solde du compte 1000 est : 4800000
(my_venv) root@ubuntu-fs:/usr/lib/cgi-bin#
```

- Supprimons tous les enregistrements de la table banque avec la commande : **truncate banque;**

```
mysql> truncate banque;
Query OK, 0 rows affected (0,14 sec)

mysql> █
```

- Créons un script python avec flask,requests et jsonify pour créer des comptes bancaires, lister, effectuer des versements , des retraits et des transferts, consulter et modifier le solde et supprimer des enregistrements :

```
GNU nano 4.8banque.py
from flask import Flask, request, jsonify
from flask_restx import Api, Resource
import apicrud

banque = Flask(__name__)
client = Api(banque)

@client.route('/creation/<prenom>/<nom>/<code>/<numcompte>/<solde>')
class Creation(Resource):
    def post(self, prenom, nom, code, numcompte, solde):
        crud.creation(prenom, nom, code, numcompte, solde)
        return jsonify({"message": f"Le compte {numcompte} a été ajouté avec succès"})

@client.route("/listecompte")
class Voir(Resource):
    def get(self):
        resultat = crud.listecompte()
        return jsonify(resultat)

@client.route('/consultation/<numcompte>/<code>')
class Consultation(Resource):
    def get(self, numcompte, code):
        solde = crud.consultation(numcompte, code)
        if solde is not None:
            return jsonify({"message": f"Le solde du compte {numcompte} est : {solde} FCFA"})
        else:
            return jsonify({"message": f"compte n*{numcompte} n'exite pas ou le mot de passe est incorrect"})

@client.route('/versement/<numcompte>/<code>/<montant>')
class Versement(Resource):
    def post(self, numcompte, code, montant):
        versement = crud.versement(numcompte, code, montant)
        if versement:
            return jsonify({"message": f"Versement de {montant} effectué avec succès sur le compte {numcompte}"})
        else:
            return jsonify({"message": f"Erreur lors du versement sur le compte {numcompte}"})

@client.route('/retrait/<numcompte>/<code>/<montant>')
class Retrait(Resource):
    def post(self, numcompte, code, montant):
        retrait = crud.retrait(numcompte, code, montant)
        if retrait:
            return jsonify({"message": f"Retrait de {montant} FCFA effectué avec succès sur le compte {numcompte}"})
        else:
            return jsonify({"message": f"Erreur lors du retrait sur le compte {numcompte}"})

@client.route('/transfert/<numcompte_donneur>/<numcompte_receveur>/<montant>')
class Transfert(Resource):
    def post(self, numcompte_donneur, numcompte_receveur, montant):
        transfert = crud.transfert(numcompte_donneur, numcompte_receveur, montant)
        if transfert:
            return jsonify({"message": f"Transfert reussie"})
        else:
            return jsonify({"message": f"Transfert impossible"})

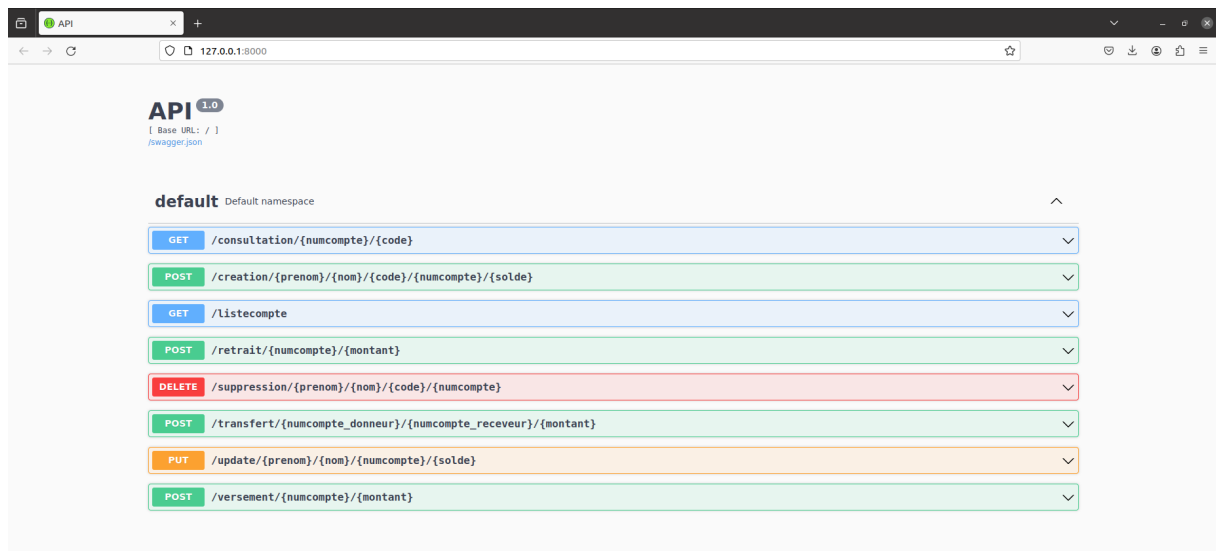
@client.route('/suppression/<prenom>/<nom>/<numcompte>/<code>')
class Suppression(Resource):
    def delete(self, prenom, nom, numcompte, code):
        suppression, message = crud.suppression(prenom, nom, numcompte, code)
        if suppression:
            return jsonify({"message": f"Le compte de {numcompte} est supprime"})
        else:
            return jsonify({"message": f"Suppression impossible"})

if __name__ == '__main__':
    banque.run(host="0.0.0.0", port=8000)
```

- Exécutons le script :

```
root@ubuntu-fs:/usr/lib/cgi-bin# python3 banque.py
* Serving Flask app 'banque'
* Debug mode: off
WARNING: This is a development server. Do not use it in production.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8000
* Running on http://192.168.1.237:8000
```

- Accédons à l'interface de l'api dans le navigateur en tapant :
http://127.0.0.1:8000



- Ajoutons des comptes bancaires (Bamba Wade, Samuel Ouya et Falilou Sarr)

Insertion de Bamba Wade :

POST /creation/{prenom}/{nom}/{code}/{numcompte}/{solde}

Parameters
Cancel

Name	Description
prenom * required string (path)	Bamba
nom * required string (path)	Wade
code * required string (path)	2407
numcompte * required string (path)	1000
solde * required string (path)	100000

Execute
Clear

Execute

Clear

Responses

Response content typeapplication/json

Curl

```
curl -X 'POST' \
'http://192.168.1.180:5000/creation/Bamba/Wade/2407/1000/100000' \
-H 'accept: application/json' \
-d ''
```

Request URL

```
http://192.168.1.180:5000/creation/Bamba/Wade/2407/1000/100000
```

Server response

Code

Details

200

Response body

```
{
  "message": "Le compte 1000 a été ajouté avec succès "
}
```

Download

Response headers

```
connection: close
content-length: 75
content-type: application/json
date: Thu,30 Nov 2023 12:26:57 GMT
server: Werkzeug/2.3.8 Python/3.8.10
```

Responses

Code

Description

200

Success

Insertion de Samuel Ouya :

POST /creation/{prenom}/{nom}/{code}/{numcompte}/{solde}

Cancel

Parameters

Name

Description

prenom * required

string

(path)

Samuel

nom * required

string

(path)

Ouya

code * required

string

(path)

1234

numcompte * required

string

(path)

1002

solde * required

string

(path)

45000

Execute

Clear

Responses

Response content typeapplication/json

Curl

```
curl -X 'POST' \
'http://192.168.1.180:5000/creation/Samuel/Ouya/1234/1002/45000' \
-H 'accept: application/json' \
-d ''
```

Request URL

```
http://192.168.1.180:5000/creation/Samuel/Ouya/1234/1002/45000
```

Server response

Code

Details

200

Response body

```
{
  "message": "Le compte 1002 a été ajouté avec succès "
}
```

Download

Insertion de Falilou Sarr :

POST /creation/{prenom}/{nom}/{code}/{numcompte}/{solde}

Parameters Cancel

Name	Description
prenom * required string (path)	Fallou
nom * required string (path)	Sarr
code * required string (path)	1301
numcompte * required string (path)	1001
solde * required string (path)	500000

Execute Clear

Responses Response content type: application/json

Curl

```
curl -X 'POST' \
  'http://192.168.1.180:5000/creation/Fallou/Sarr/1301/1001/500000' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://192.168.1.180:5000/creation/Fallou/Sarr/1301/1001/500000
```

Server response

Code	Details
200	Response body <pre>{ "message": "Le compte 1001 a été ajouté avec succès " }</pre> Download

- Vérifions dans notre base de donnée si les comptes ont bien été ajoutés :

```
mysql> select * from client;
+----+-----+-----+-----+-----+-----+
| id | prenom | nom  | code | numcompte | solde |
+----+-----+-----+-----+-----+-----+
| 1  | Bamba  | Wade | 2407 | 1000      | 100000 |
| 3  | Samuel | Ouya  | 1234 | 1002      | 45000  |
| 5  | Falilou | Sarr  | 1301 | 1001      | 500000 |
+----+-----+-----+-----+-----+-----+
3 rows in set (0,00 sec)

mysql> 
```


- Effectuons un versement de 50000 sur le compte de Falilou Sarr :

POST /versement/{numcompte}/{code}/{montant}

Parameters Cancel

Name	Description
numcompte * required string (path)	1001
code * required string (path)	1301
montant * required string (path)	50000

Execute **Clear**

Responses Response content type: application/json

Curl

```
curl -X 'POST' \
'http://192.168.1.180:5000/versement/1001/1301/50000' \
-H 'accept: application/json' \
-d ''
```

Request URL

```
http://192.168.1.180:5000/versement/1001/1301/50000
```

Server response

Code	Details
200	Response body <pre>{ "message": "Versement de 50000 effectué avec succès sur le compte 1001" }</pre> Download

- Vérifions si le versement a bien été fait sur le compte dans notre base de donnée :

```
mysql> select * from client;
```

id	prenom	nom	code	numcompte	solde
1	Bamba	Wade	2407	1000	100000
3	Samuel	Ouya	1234	1002	45000
5	Falilou	Sarr	1301	1001	500000

3 rows in set (0,00 sec)

avant

```
mysql> select * from client;
```

id	prenom	nom	code	numcompte	solde
1	Bamba	Wade	2407	1000	100000
3	Samuel	Ouya	1234	1002	45000
5	Falilou	Sarr	1301	1001	550000

3 rows in set (0,00 sec)

- Effectuons un retrait de 25000 sur le compte de Bamba Wade :

POST /retrait/{numcompte}/{code}/{montant}

Parameters Cancel

Name	Description
numcompte * required string (path)	1000
code * required string (path)	2407
montant * required string (path)	25000

Execute **Clear**

Responses Response content type: application/json

Curl

```
curl -X 'POST' \
  'http://192.168.1.180:5000/retrait/1000/2407/25000' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://192.168.1.180:5000/retrait/1000/2407/25000
```

Server response

Code	Details
200	Response body <pre>{ "message": "Retrait de 25000 effectué avec succès sur le compte 1000" }</pre> Download

- Vérifions si le retrait est bien fait dans la base de donnée :

```
mysql> select * from client;
+----+-----+-----+-----+-----+-----+
| id | prenom | nom  | code | numcompte | solde |
+----+-----+-----+-----+-----+-----+
| 1  | Bamba  | Wade | 2407 | 1000      | 100000 |
| 3  | Samuel | Ouya  | 1234 | 1002      | 45000  |
| 5  | Falilou | Sarr  | 1301 | 1001      | 550000 |
+----+-----+-----+-----+-----+-----+
3 rows in set (0,00 sec)

mysql> select * from client;
+----+-----+-----+-----+-----+-----+
| id | prenom | nom  | code | numcompte | solde |
+----+-----+-----+-----+-----+-----+
| 1  | Bamba  | Wade | 2407 | 1000      | 75000  |
| 3  | Samuel | Ouya  | 1234 | 1002      | 45000 |
| 5  | Falilou | Sarr  | 1301 | 1001      | 550000 |
+----+-----+-----+-----+-----+-----+
3 rows in set (0,00 sec)
```

- Effectuons un transfert de 100000 du compte de Falilou Sarr vers Samuel Ouya :

POST /transfer/{numcompte_donneur}/{numcompte_receveur}/{montant}

Parameters Cancel

Name	Description
numcompte_donneur * required string (path)	1001
numcompte_receveur * required string (path)	1002
montant * required string (path)	100000

Execute Clear

Responses Response content type: application/json

Curl

```
curl -X 'POST' \
  'http://192.168.1.180:5000/transfer/1001/1002/100000' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://192.168.1.180:5000/transfer/1001/1002/100000
```

Server response

Code	Details
200	Response body <pre>{ "message": "Transfert reussie" }</pre> Download

- Vérifions les soldes des comptes dans la base de donnée :

```
mysql> select * from client;
+----+-----+-----+-----+-----+-----+-----+
| id | prenom | nom   | code | numcompte | solde |
+----+-----+-----+-----+-----+-----+
| 1  | Bamba  | Wade | 2407 | 1000      | 75000 |
| 3  | Samuel | Ouya   | 1234 | 1002      | 45000 |
| 5  | Falilou | Sarr   | 1301 | 1001      | 550000 |
+----+-----+-----+-----+-----+-----+
3 rows in set (0,00 sec)

mysql> select * from client;
+----+-----+-----+-----+-----+-----+
| id | prenom | nom   | code | numcompte | solde |
+----+-----+-----+-----+-----+-----+
| 1  | Bamba  | Wade | 2407 | 1000      | 75000 |
| 3  | Samuel | Ouya   | 1234 | 1002      | 145000 |
| 5  | Falilou | Sarr   | 1301 | 1001      | 450000 |
+----+-----+-----+-----+-----+-----+
3 rows in set (0,00 sec)

mysql>
```

- Consultons le solde du compte bancaire de Bamba Wade avec 1000 comme numéro de compte :

GET /consultation/{numcompte}/{code}

Parameters Cancel

Name	Description
numcompte * required string (path)	1000
code * required string (path)	2407

Execute **Clear**

Responses Response content type application/json

Curl

```
curl -X 'GET' \
'http://192.168.1.180:5000/consultation/1000/2407' \
-H 'accept: application/json'
```

Request URL

```
http://192.168.1.180:5000/consultation/1000/2407
```

Server response

Code	Details
200	Response body <pre>{ "message": "Le solde du compte 1000 est : 75000" }</pre>Download Response headers

- Supprimons le compte bancaire de Bamba Wade avec 1000 comme numéro de compte :

DELETE /suppression/{prenom}/{nom}/{numcompte}/{code}

Parameters Cancel

Name	Description
prenom * required string (path)	Bamba
nom * required string (path)	Wade
numcompte * required string (path)	1000
code * required string (path)	2407

Execute **Clear**

Responses Response content type application/json

Curl

```
curl -X 'DELETE' \
'http://192.168.1.180:5000/suppression/Bamba/Wade/1000/2407' \
-H 'accept: application/json'
```

Request URL

```
http://192.168.1.180:5000/suppression/Bamba/Wade/1000/2407
```

Server response

Code	Details
200	Response body <pre>{ "message": "Le compte de 1000 est supprimé avec succès" }</pre>Download

- Vérifions que le compte a bien été supprimé dans la base de donnée :

```
mysql> select * from client;
+-----+-----+-----+-----+-----+-----+
| id | prenom | nom | code | numcompte | solde |
+-----+-----+-----+-----+-----+-----+
| 3 | Samuel | Ouya | 1234 | 1002 | 145000 |
| 5 | Falilou | Sarr | 1301 | 1001 | 450000 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0,00 sec)

mysql>
```

- Modifions le solde du compte de Samuel Ouya :

PUT /update/{prenom}/{nom}/{numcompte}/{solde}

Parameters

Name	Description
prenom * required string (path)	Samuel
nom * required string (path)	Ouya
numcompte * required string (path)	1002
solde * required string (path)	150000

Execute

Clear

Responses

Response content type application/json

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/update/Samuel/Ouya/1002/150000' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/update/Samuel/Ouya/1002/150000
```

Server response

Code	Details
200	Response body <pre>{ "message": "le solde du compte 1002 a ete mise a jour" }</pre> Download

- Vérifions si le solde a été mise à jour :

```
mysql> select * from client;
+-----+-----+-----+-----+-----+-----+
| id | prenom | nom | code | numcompte | solde |
+-----+-----+-----+-----+-----+-----+
| 3 | Samuel | Ouya | 1234 | 1002 | 150000 |
| 5 | Falilou | Sarr | 1301 | 1001 | 450000 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0,00 sec)

mysql> █
```